

Shikaku

Sofia Guerra Jiménez^a

^a*Pontificia Universidad Javeriana, Facultad de Ingeniería, Departamento de Ingeniería de Sistemas*

Abstract

En este documento se presenta la escritura formal de los algoritmos necesarios para implementar un solucionador automático del juego Shikaku. Se describen el análisis del problema, la clasificación de su complejidad computacional, el diseño de entradas y salidas, los algoritmos formales, su estrategia algorítmica, su análisis de complejidad y su invariante.

Keywords: Shikaku, algoritmo, NP-Completo, combinatoria, heurística, problema de decisión.

1. Análisis del problema

Shikaku es un rompecabezas que se desarrolla en un tablero de $W \times H$ celdas. En el tablero hay celdas que contienen un número entero positivo. La suma de estos números equivale al área del rompecabezas, es decir, $W \times H$. El objetivo del juego es dividir el tablero en rectángulos de manera que cada rectángulo contenga exactamente un número, que definirá su área. Todas las celdas pertenecen exactamente a un rectángulo.

1.1. Clasificación del problema

El problema de Shikaku es un problema de decisión. Se puede basar en la siguiente pregunta: ¿Existe una partición válida para el tablero? O al escoger un rectángulo, ¿El rectángulo puede ir en esa posición?

El problema se puede clasificar como NP-Completo. Para que un problema sea NP-Completo, debe ser al menos tan difícil como un problema NP, y debe ser verificable en tiempo polinomial.

La solución del tablero de Shikaku puede ser verificable en tiempo polinomial, recorriendo el tablero una vez.

1.2. Análisis previo de complejidad: Fuerza bruta vs. Heurística

Antes de empezar a implementar la estrategia algorítmica, se debe saber cuál será la aproximación que se va a tomar. Se puede realizar el algoritmo por fuerza

bruta, o buscar una heurística. Para tomar esta decisión, primero se calcula la complejidad de la fuerza bruta. Si es demasiado alta, vale la pena buscar una heurística, de lo contrario, puedo quedarme con el algoritmo por fuerza bruta.

1.2.1. Fuerza bruta

Sea $T = W \times H$ el número total de celdas, n la cantidad de números enteros en el tablero, y k_i el valor de cada uno. Por definición del problema, la suma de todos esos valores cubre exactamente el tablero:

$$T = k_1 + k_2 + k_3 + \dots + k_n$$

Para analizar el problema por fuerza bruta, podemos tomar una aproximación por combinatoria.

Para comenzar, de las T celdas disponibles, escogemos el primer valor k_1 , y buscamos todas las combinaciones posibles de rectángulos válidos con k_1 celdas, y así sucesivamente.

Sin tener en cuenta ninguna de las restricciones del rompecabezas, el problema combinatorio se ve así:

$$C_{total} = C(T, k_1) \times C(T - k_1, k_2) \times C(T - k_1 - k_2, k_3) \times \dots \times C(k_n, k_n)$$

Para analizar si una fuerza bruta es viable, debo ver que tan posible es implementarla en la práctica. Con un tablero de $9 \times 9 = 81$ celdas y valores $k_i = 5$, con $n = 15$ números:

$$C_{total} = C(81, 5) \times C(76, 5) \times C(71, 5) \times \dots \approx 10^{26}$$

Esta cantidad es muy difícil de trabajar en la práctica, lo que justifica buscar una heurística. (Aunque en este proyecto se implementará una fuerza bruta con backtracking por facilidad).

1.2.2. Heurística

La heurística propuesta es MRV (Minimum Remaining Values). En cada nivel del backtracking se recalcula cuántos rectángulos válidos quedan disponibles para cada número no asignado en el estado actual del tablero, y se elige el que tenga menos opciones. Al colocar primero el número más restringido en cada momento, se detectan los caminos sin salida antes de llegar a niveles más profundos del árbol de búsqueda. En el peor caso la complejidad sigue siendo la misma que la fuerza bruta, pero en casos bien contruidos, se puede reducir bastante el árbol.

2. Diseño del problema

2.1. Entradas

- $W \in \mathbb{N}$: Ancho del tablero (columnas)
- $H \in \mathbb{N}$: Alto del tablero (filas)

- Matriz de enteros $W \times H$, donde cada celda puede ser un número puede contener un número entero positivo k , o -1 si no tiene valor. Tenemos la garantía que la suma de los números enteros positivos del tablero serán iguales a $W \times H$, o el área del tablero.

2.2. Salidas

El algoritmo devuelve una asignación completa del tablero, o una partición en rectángulos tal que cada rectángulo contiene exactamente un número, y su área es igual a ese número. Formalmente, la salida es un conjunto de tuplas (f, c, h, w) donde f es la fila de la esquina superior izquierda, c la columna, h el alto y w el ancho del rectángulo asignado.

3. Algoritmo de solución

Para este proyecto, el solucionador se implementa de dos maneras. Fuerza bruta (Algoritmo 2) y heurística Minimum Remaining Value (MRV) (Algoritmo 3). Ambas comparten el esquema de backtracking, pero difieren en el orden en que eligen el siguiente número a asignar.

3.1. Estrategia algorítmica

Fuerza Bruta: El algoritmo itera sobre los números del tablero en el orden en que fueron leídos. Para cada número no asignado, genera todos los rectángulos válidos posibles (que lo contienen, caben en el tablero, no encierran otro número, y no solapan con ningún rectángulo ya asignado). Prueba cada candidato, marca las celdas, y recurre al siguiente número. Si ningún candidato lleva a solución, deshace la asignación (backtracking) y prueba el siguiente.

Heurística MRV: En cada nivel del backtracking, en lugar de seguir un orden fijo, se recalcula en el estado actual del tablero cuántos rectángulos válidos tiene cada número no asignado, y se elige el que tenga menor cantidad (Minimum Remaining Values). Si durante este recálculo se detecta que algún número aún no asignado ya no tiene ningún rectángulo válido posible, se descarta la rama inmediatamente sin explorarla. Esto reduce el árbol de búsqueda de forma significativa en la práctica.

Algorithm 1 GenerarValidos(tablero, f, c, val)

Require: *tablero*: estado actual con celdas y regiones**Require:** (f, c) : coordenadas del número**Require:** *val*: valor del número (área del rectángulo)**Ensure:** *validos*: lista de rectángulos $(fila, col, alto, ancho)$ que cumplen las condiciones

```

1: procedure GENERARVALIDOS(tablero, f, c, val)
2:    $H \leftarrow \text{tablero.filas}$ ,  $W \leftarrow \text{tablero.columnas}$ 
3:    $validos \leftarrow \emptyset$ 
4:   for  $alto \leftarrow 1$  to  $val$  do
5:     if  $val \bmod alto \neq 0$  then continue
6:     end if
7:      $ancho \leftarrow val / alto$ 
8:     if  $ancho > W$  then continue
9:     end if
10:     $fila_{min} \leftarrow \max(0, f - alto + 1)$ 
11:     $fila_{max} \leftarrow \min(H - alto, f)$ 
12:     $col_{min} \leftarrow \max(0, c - ancho + 1)$ 
13:     $col_{max} \leftarrow \min(W - ancho, c)$ 
14:    for  $fila \leftarrow fila_{min}$  to  $fila_{max}$  do
15:      for  $col \leftarrow col_{min}$  to  $col_{max}$  do
16:         $descartado \leftarrow \text{False}$ 
17:        for  $i \leftarrow fila$  to  $fila + alto - 1$  do
18:          for  $j \leftarrow col$  to  $col + ancho - 1$  do
19:            if  $\text{tablero.celdas}[i][j] > 0$  and  $(i, j) \neq (f, c)$  then
20:               $descartado \leftarrow \text{True}$ 
21:            end if
22:            if  $\text{tablero.regiones}[i][j] \geq 0$  then
23:               $descartado \leftarrow \text{True}$ 
24:            end if
25:          end for
26:        end for
27:        if not  $descartado$  then
28:           $validos \leftarrow validos \cup \{(fila, col, alto, ancho)\}$ 
29:        end if
30:      end for
31:    end for
32:  end for
33:  return  $validos$ 
34: end procedure

```

Algorithm 2 Principal_FB(tablero)

Require: *tablero*: estado inicial con celdas y dimensiones**Ensure:** **true** si se encontró solución, **false** en caso contrario

```

1: procedure PRINCIPAL_FB(tablero)
2:   for each  $(i, j)$  in tablero.regiones do regiones[i][j]  $\leftarrow -1$  end for
3:   rectangulos  $\leftarrow \emptyset$ 
4:   numeros  $\leftarrow \emptyset$ 
5:   for  $i \leftarrow 0$  to  $H - 1$  do
6:     for  $j \leftarrow 0$  to  $W - 1$  do
7:       if tablero.celdas[i][j]  $> 0$  then
8:         numeros  $\leftarrow numeros \cup \{(i, j, celdas[i][j])\}$ 
9:       end if
10:    end for
11:  end for
12:  if numeros =  $\emptyset$  then return false
13:  end if
14:  posibilidades  $\leftarrow \emptyset$ 
15:  for  $k \leftarrow 0$  to  $|numeros| - 1$  do
16:     $(f, c, val) \leftarrow numeros[k]$ 
17:    posibilidades[k]  $\leftarrow$  GENERARVALIDOS(tablero, f, c, val)
18:  end for
19:  return BACKTRACKINGFB(tablero, numeros, posibilidades, 0)
20: end procedure

```

Algorithm 3 BacktrackingFB(tablero, numeros, posibilidades, idx)

Require: *tablero*: estado actual
Require: *numeros*: lista de números en orden fijo
Require: *posibilidades*: rectángulos candidatos para cada número
Require: *idx*: índice del número actual a asignar
Ensure: **true** si se encontró solución, **false** en caso contrario

```

1: procedure BACKTRACKINGFB(tablero, numeros, posibilidades, idx)
2:   if  $idx = |numeros|$  then return true
3:   end if
4:   for each  $R \in posibilidades[idx]$  do
5:     if ESVALIDO(tablero, R) then
6:        $id \leftarrow |tablero.rectangulos|$ 
7:        $rectangulos \leftarrow rectangulos \cup \{R\}$ 
8:       MARCAR(tablero, R, id)
9:       if BACKTRACKINGFB(tablero, numeros, posibilidades,  $idx + 1$ )
10:      then return true
11:     end if
12:      $rectangulos \leftarrow rectangulos \setminus \{R\}$ 
13:     DESMARCAR(tablero, R)
14:   end for return false
15: end procedure

```

Algorithm 4 Principal_H(tablero)

Require: *tablero*: estado inicial con celdas y dimensiones
Ensure: **true** si se encontró solución, **false** en caso contrario

```

1: procedure PRINCIPAL_H(tablero)
2:   for each  $(i, j)$  in tablero.regiones do  $regiones[i][j] \leftarrow -1$  end for
3:    $rectangulos \leftarrow \emptyset$ 
4:    $numeros \leftarrow \emptyset$ 
5:   for  $i \leftarrow 0$  to  $H - 1$  do
6:     for  $j \leftarrow 0$  to  $W - 1$  do
7:       if tablero.celdas $[i][j] > 0$  then
8:          $numeros \leftarrow numeros \cup \{(i, j, celdas[i][j])\}$ 
9:       end if
10:    end for
11:  end for
12:  if  $numeros = \emptyset$  then return false
13:  end if
14:   $asignados \leftarrow [False] \times |numeros|$ 
15:  return BACKTRACKINGMRV(tablero, numeros, asignados,  $|numeros|$ )
16: end procedure

```

Algorithm 5 BacktrackingMRV(tablero, numeros, asignados, restantes)

Require: *tablero*: estado actual**Require:** *numeros*: lista de todos los números del tablero**Require:** *asignados*: vector booleano que indica qué números ya tienen rectángulo**Require:** *restantes*: cantidad de números aún no asignados**Ensure:** **true** si se encontró solución, **false** en caso contrario

```

1: procedure BACKTRACKINGMRV(tablero, numeros, asignados, restantes)
2:   if restantes = 0 then return true
3:   end if
4:   mejor  $\leftarrow -1$ , mejorCount  $\leftarrow -1$ , mejorValidos  $\leftarrow \emptyset$ 
5:   for k  $\leftarrow 0$  to  $|numeros| - 1$  do
6:     if asignados[k] then continue
7:     end if
8:     (f, c, val)  $\leftarrow numeros[k]$ 
9:     v  $\leftarrow$  GENERARVALIDOS(tablero, f, c, val)
10:    if v =  $\emptyset$  then return false
11:    end if
12:    if mejor =  $-1$  or  $|v| < mejorCount$  then
13:      mejor  $\leftarrow k$ 
14:      mejorCount  $\leftarrow |v|$ 
15:      mejorValidos  $\leftarrow v$ 
16:    end if
17:  end for
18:  if mejor =  $-1$  then return false
19:  end if
20:  asignados[mejor]  $\leftarrow$  True
21:  for each R  $\in$  mejorValidos do
22:    id  $\leftarrow |tablero.rectangulos|$ 
23:    rectangulos  $\leftarrow rectangulos \cup \{R\}$ 
24:    MARCAR(tablero, R, id)
25:    if BACKTRACKINGMRV(tablero, numeros, asignados, restantes - 1)
then return true
26:    end if
27:    rectangulos  $\leftarrow rectangulos \setminus \{R\}$ 
28:    DESMARCAR(tablero, R)
29:  end for
30:  asignados[mejor]  $\leftarrow$  False return false
31: end procedure

```

3.2. Análisis de complejidad

Sea n la cantidad de números en el tablero y p_i el número de rectángulos candidatos para el i -ésimo número. En el peor caso, ambos algoritmos exploran

$$\prod_{i=1}^n p_i \quad (\text{es decir, } p_1 \times p_2 \times \cdots \times p_n)$$

combinaciones, lo que representa el producto de las cantidades de opciones para cada número. Para fuerza bruta, p_i se calcula con el tablero vacío. El número de divisores de k_i acota el número de dimensiones posibles, y la posición puede variar en hasta k_i lugares, por lo que $p_i = O(k_i \cdot d(k_i))$, donde $d(k_i)$ es el número de divisores de k_i .

Para la heurística MRV, aunque la complejidad del peor caso es la misma (el problema es NP-completo), el árbol explorado en la práctica es mucho menor. Antes de llegar a los niveles más profundos, se identifican conflictos, para elegir siempre el número más restringido y reducir la cantidad de ramas que se exploran por nivel.

3.3. Invariante del algoritmo

Durante todo el backtracking, las celdas que no están marcadas como -1 en la matriz de regiones constituyen una colección de rectángulos válidos. Cada rectángulo contiene exactamente un número, su área es igual al valor de ese número, y ninguna celda pertenece a más de un rectángulo.

La búsqueda termina cuando todos los números han sido asignados a un rectángulo, o cuando se llega a una contradicción (algún número no asignado no tiene ningún rectángulo candidato válido en el estado actual del tablero).

3.4. Notas de implementación

A continuación se presentan los requisitos y consideraciones necesarios para implementar el algoritmo:

- Lenguaje con soporte para estructuras de datos de dos dimensiones para representar celdas y regiones.
- El tablero mantiene dos matrices separadas. Hay celdas (valores fijos, que no cambian durante la búsqueda) y regiones (asignación de rectángulos, modificada durante el backtracking).
- El backtracking requiere una operación para deshacer una asignación de manera eficiente. Al revertir una asignación, es suficiente poner a -1 las celdas del rectángulo desmarcado, sin necesidad de reconstruir toda la matriz de regiones.
- Para la heurística MRV, generarValidos combina en una sola vuelta la verificación por otro número y la verificación por celda ya ocupada, evitando dos recorridos separados del rectángulo candidato.

- El tipo de dato para las celdas debe ser entero con signo, ya que `regiones[i][j] = -1` indica una celda libre.